

W4RYA

NETWORK FLOW ANALYZER · ATTACK & DEFENSE CTF

Manual de uso completo.
Operación durante competencia en vivo.

V0.4.0 · HARD FORK OF TULIP · GPL-3.0

NO AI AT RUNTIME

| CONTENIDOS

01	Sobre w4rya	3
02	Instalación y primer arranque	3
03	Auth y roles	5
04	Configuración inicial	6
05	Vista principal: FlowList y filtros	7
06	Inspeccionar un flow	8
07	Test de exploits + script para farm	9
08	Suricata rules + quick-block	10
09	Attack timeline	12
10	War-room (modo TV)	12
11	Notas y alertas	13
12	Audit log	14
13	Backups	15
14	Tests automatizados	16
15	Troubleshooting	16
16	Atajos de teclado	18

SOBRE W4RYA

CAPÍTULO 01

w4rya es un analizador de flujos de red diseñado para usarse durante competencias Attack/Defense (A/D) de CTF. Ingiere los pcaps que captura tu vulnbox, los descompone en flows, los enriquece con tags (incluyendo alerts de Suricata si está activo), y los expone en una UI web pensada para que un equipo opere en simultáneo: buscar el ataque que acaba de robar un flag, escribir una regla Suricata para bloquearlo, y rebotar el mismo exploit a los otros teams.

STACK

COMPONENTE	TECNOLOGÍA	ROL
Ingestor	Go (gopacket)	Lee pcaps, reconstruye flows, escribe a Timescale
API	Python Flask	Query + auth + config + rules + replay
Frontend	React + Vite + Tailwind	Operación en vivo
DB	TimescaleDB / Postgres	Persistencia + extensión C custom
Suricata	Container opcional	IDS/IPS, eve.json → enricher

RESTRICCIÓN PERMANENTE

Este tool **NUNCA** llama a un servicio de IA/LLM en tiempo de ejecución. Las competencias A/D donde está pensado correr prohíben uso de IA durante el juego. Toda la clasificación es determinista: regex, signatures de Suricata, heurísticas estadísticas. PRs que agreguen llamadas a OpenAI/Anthropic/cualquier API de LLM serán rechazados.

BASADO EN

Hard fork de **Tulip** (github.com/OpenAttackDefenseTools/tulip) que a su vez es un fork de **flower** (github.com/secgroup/flower). Toda la arquitectura del ingestor y la base del frontend vienen de Tulip; w4rya agrega auth con roles, exploit testing, war-room mode, audit log, polish de UI dark hacking, y hardening para uso en vivo.

INSTALACIÓN Y PRIMER ARRANQUE

CAPÍTULO 02

REQUISITOS

- **Docker** 24+ con `docker compose` v2
- 4 GB RAM mínimo recomendado (8 GB para CTFs con mucho tráfico)

- Tu usuario en el grupo `docker` para no usar sudo

CLONAR Y CONFIGURAR

```
git clone https://github.com/JCaleb2001/w4rya.git
cd w4rya

cp .env.example .env
# editar .env: poner un W4RYA_SECRET_KEY (ver más abajo)
```

El secret key es obligatorio. Génalo con:

```
openssl rand -hex 32
```

Copialo dentro de `.env` en la línea `W4RYA_SECRET_KEY=...`. Sin este valor el api se niega a arrancar.

LEVANTAR EL STACK

```
docker compose up -d --build
```

Esto levanta 6 containers: `timescale`, `api`, `frontend`, `assembler`, `enricher`, `flagids`. La primera vez tarda ~3-5 min compilando la extensión Postgres y el ingestor Go.

VERIFICAR QUE ARRANCÓ

```
docker compose ps          # los 6 containers en Up
curl http://localhost:3001/api/healthz  # → {"db":"up","ok":true}
```

CREAR EL USUARIO ADMIN INICIAL

```
docker compose run --rm api python /app/auth/add_user.py <username>
```

Prompts interactivos para password (no se ve mientras tipeas). El primer usuario que crees queda con role `admin` por back-compat (los entries sin `role:` default a admin). Usuarios futuros via el mismo script defaultean a `viewer`; usá `--role admin|operator|viewer` para especificar.

FOOTGUN

NO uses `sudo` con ese comando. El file `auth/users.yaml` queda root-owned en el host. Si ya lo hiciste: `sudo chown -R $USER:$USER auth/`.

ABRIR LA UI

Navegá a `http://localhost:3001`. Login con el usuario que acabás de crear. Te redirige al dashboard.

VARIANTE CON SURICATA

Para activar IDS/IPS, levántala con `docker compose -f docker-compose-suricata.yml up -d --build`. Necesitas también `SURICATA_DIR_HOST` en `.env` apuntando a un directorio host donde Suricata escribe logs.

AUTH Y ROLES

CAPÍTULO 03

Tres tiers de permisos, en orden ascendente:

ROLE	LO QUE PUEDE HACER
VIEWER	Browse flows, ver dashboard, war-room. Postear notas. Login/logout.
OPERATOR	Todo lo anterior + editar Suricata rules, reload Suricata, test-exploit replay, quick-block IP, star flow.
ADMIN	Todo + <code>/config</code> (services/teams/game vars) + <code>/audit</code> .

La UI esconde los botones que no podés usar; el backend de todos modos los protege con 403.

CREAR USUARIOS

```
docker compose run --rm api python /app/auth/add_user.py alice --role operator
docker compose run --rm api python /app/auth/add_user.py bob --role viewer
docker compose restart api # opcional, mtime cache también lo agarra
```

ROTAR PASSWORD

Mismo comando con el mismo username: sobrescribe el hash y el role.

BORRAR UN USUARIO

Editar `auth/users.yaml` a mano, eliminar el bloque correspondiente, `docker compose restart api`.

ROTAR EL SECRET KEY (INVALIDA TODAS LAS SESIONES)

```
NEW=$(openssl rand -hex 32)
sed -i "s|^W4RYA_SECRET_KEY=.*|W4RYA_SECRET_KEY=\"$NEW\"|" .env
docker compose restart api
```

RATE LIMITING

El endpoint `/login` tiene un token bucket en memoria: **5 intentos fallidos en 5 minutos** por cada par (ip, username) → 429 con `retry_after_sec`. Cada unicorn worker tiene su propio contador, así que en práctica con 3 workers el techo efectivo es ~15 intentos. Suficiente para frenar scripting básico.

SESIÓN COOKIE

Nombre `w4rya_session`, `httpOnly`, `SameSite=Lax`, 7 días de vida. Para HTTPS poné `W4RYA_COOKIE_SECURE=1` en `.env` antes de servir por TLS.

CONFIGURACIÓN INICIAL

CAPÍTULO 04

Antes de empezar a usar w4rya en un CTF real, necesitás configurar 3 cosas en `/config` (solo admin):

TAB "GAME"

Variables del juego que se aplican al ingestor y a la UI:

VAR	QUÉ ES
flag_regex	Patrón regex que matchea el formato de flag del CTF. Ej: <code>[A-Z0-9]{31}=</code> . Validado al escribir (rechaza regex inválido y patrones ReDoS).
tick_start	Timestamp ISO8601 UTC del inicio del primer tick. Ej: <code>2024-11-30T13:00:00Z</code> .
tick_length	Duración de un tick en ms. Típico 180000 (3 min) o 240000 (4 min).
flag_lifetime	Cuántos ticks vive un flag antes de expirar. Default 5.
vm_ip	IP de tu vulnbox dentro de la red del juego.
team_id	Número/ID de tu equipo.
visualizer_url	Opcional: URL del visualizer del CTF (Faust style).
bpf	Opcional: BPF filter para limitar captura.

RESTART REQUIRED

`flag_regex` y `bpf` también los lee el container `assembler` al boot. Si los cambiás en vivo, la UI los aplica al instante pero el ingestor sigue con el valor viejo hasta que reinicies con `docker compose restart assembler`.

TAB "SERVICES"

Lista de servicios de TU equipo (los que el ingestor ve cuando rotura el tráfico que cae en tu vulnbox). Cada entry: `name`, `ip`, `port`, `notes`. Estos se usan para:

- Etiquetar flows con el service-tag correspondiente
- Mostrar chips de filtrado en la sidebar del FlowList
- Stats per-service (flows, attacks, flags-out en los últimos 5 ticks)

Ejemplo: si tu vulnbox es `10.10.3.1` y el servicio "BIOMarkt" corre en `:18080`, agregás:
`name=BIOMarkt ip=10.10.3.1 port=18080`.

TAB "TEAMS"

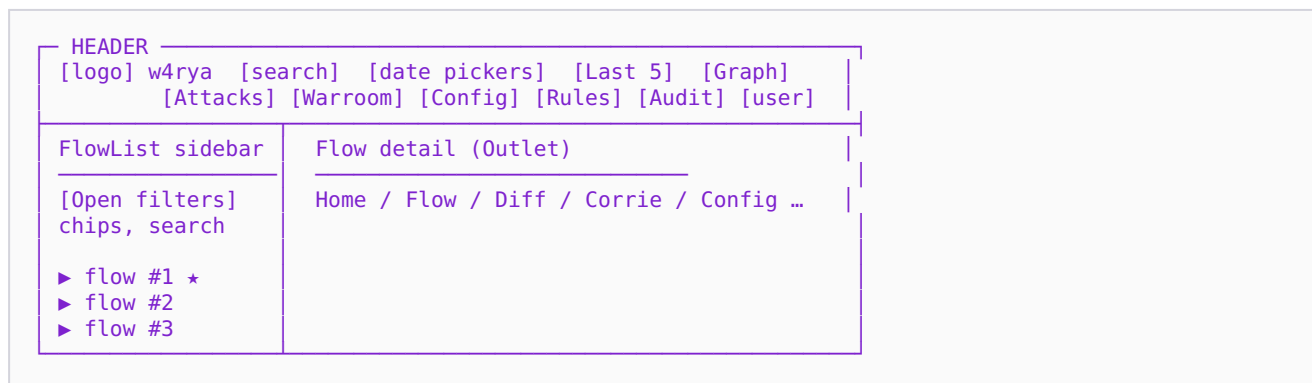
Lista de equipos enemigos. Cada entry: `name`, `ip`, `notes`. Esta tabla solo se usa por el **exploit replay**: cuando disparas "test exploit" desde un flow, w4rya enumera estos team-ips y rebota el payload contra cada uno en su puerto original.

Sin teams configurados, el modal de exploit testing va a estar vacío.

VISTA PRINCIPAL: FLOWLIST Y FILTROS

CAPÍTULO 05

El dashboard tiene tres áreas:



BUSCAR Y FILTRAR

El sidebar tiene un panel de filtros (botón "Open filters") con dos secciones:

- **Services:** chips multi-select de los services configurados. Click toggle. Cada chip muestra mini-stats (flujos/🚩 últimos 5 ticks). El borde del chip refleja presión: violeta normal, amarillo bajo alerta Suricata, rojo si perdiste flag.
- **Tags intersection:** chips de tags (incluyendo `flag-in`, `flag-out`, `starred`, etc). Tres estados: deshabilitado, incluido, excluido (negro tachado). Modo AND/OR para múltiples tags.

En el **header**:

- **Search bar**: regex case-insensitive sobre el contenido de los flows (debounced 300ms)
- **Date pickers**: tick start / end manual
- **"Last 5 ticks"**: shortcut

ICONOS DE CADA FLOW

ICON	SIGNIFICA
♥ (rojo glow)	Starred
🔗 (violeta)	Tiene parent o child (flow encadenado)
service:port	Service tag resuelto (o "unknown")
Duración >10s en rojo	Flow largo, posible interés

POLLING

El FlowList re-poolea cada 30s automáticamente. Si la pestaña está en background, el polling se pausa (visibility-aware).

INSPECCIONAR UN FLOW

CAPÍTULO 06

Click en cualquier flow del sidebar abre `/flow/<id>` en el panel derecho.

ESTRUCTURA

1. **Toolbar secundario** (sticky top): nav parent/child, selector de decoder (raw / http-decoded / etc), "Diff" para representaciones diferentes, "Copy as pwntools", "Copy as requests", "► test exploit".
2. **Suricata card** (si el flow disparó alguna regla): rule msg, rule id, action (alert / drop).
3. **Meta card**: tags, tick, service, flags-in/out, src → dst, duración. Botón **"block"** al lado del src_ip que crea regla Suricata DROP automáticamente.
4. **Notes panel**: scratchpad del equipo (ver §11).
5. **Flow items**: cada request/response del flow renderizado según el modo elegido:

MODO	CUÁNDO
Plain	Texto crudo del wire. Default.
Hex	Hexdump (formato <code>twos</code>). Para protocolos binarios.
Web	HTTP only: renderiza el body en un <code><iframe></code> sandboxed. Útil para ver una respuesta HTML como la vería un browser.

HIGHLIGHTS EN EL BODY

El texto del flow tiene dos highlights automáticos:

- El query del search bar (si está activo) en violeta translúcido
- Cualquier match del `flag_regex` en rojo translúcido (más vistoso, indica leak)

EXPORT

- **Download:** bytes crudos del flow item, archivo binario
- **Download body** (solo Web): body decodificado
- **Open in CC:** abre el contenido en CyberChef (gchq.github.io). *Nota:* este botón envía data a github.io — no clickear con flags reales si te preocupa privacidad.
- **Copy as pwntools / Copy as requests:** snippets listos para tu farm

DIFF ENTRE REPRESENTACIONES

Si un flow tiene varios decoders (ej: `raw` + `http-decoded`), podés diff entre ellos con el botón ⚡. Útil para detectar cambios en encoding (ej: gzip + raw).

ATAJOS EN FLOWVIEW

TECLA	ACCIÓN
<code>j</code> / <code>k</code>	Next / prev flow en sidebar
<code>h</code> / <code>l</code>	Prev / next item dentro del flow
<code>w</code>	Scroll al flow actual en sidebar
<code>x</code>	Toggle star
<code>m</code>	Rotar entre decoders
<code>d</code>	Diff view del flow

TEST DE EXPLOITS + SCRIPT PARA FARM

CAPÍTULO 07

El feature más A/D-específico. Workflow esperado:

1. Un equipo enemigo te explota — ves un flow en el FlowList con tag `flag-out` o una alerta Suricata.

2. Abrís el flow, identificás el payload del request enemigo.
3. Click "► **test exploit**". Se abre el modal.
4. El modal toma los items con `direction=c` (cliente → servidor) y los concatena como payload. Te muestra preview (bytes, puerto, src/dst originales).
5. Seleccionás contra qué teams disparar (default: todos los `/config/teams`). Setteás timeout (default 3s).
6. "► **replay against N target(s)**" → fan-out paralelo (max 16 workers, deadline overall 45s < unicorn -t 60s).
7. La tabla de resultados se ordena: hits primero, después OK, después errores. Cada fila muestra: team, target IP, status (✓ FLAG / ○ no flag / ! ERR), latencia, response size, conteo de flags capturadas, excerpt.
8. Click una fila → expande con el body completo y los flag values encontrados.

DESCARGAR EL EXPLOIT PARA EL FARM

El botón "↓ **download exploit script**" baja un `.py` standalone (sin dependencias externas) con:

- El payload embebido como hex literal
- La lista de teams configurados, baked in
- El flag_regex
- Un `main()` que loopea por team, abre socket, sendall, captura, regex-matchea, imprime
- Un comentario `# TODO: pipe captured into your farm/submitter` donde plug-ueás tu lógica de submission

LÍMITE DEL REPLAY ACTUAL

El payload se envía tal cual fue capturado. Funciona para exploits stateless. Si el exploit dependía de un flagid específico por-team, vas a necesitar editar el script generado para inyectar el flagid correcto (esto está en roadmap como "tokenized flagid replay").

CAPS

- MAX_TARGETS: 64 por replay
- Per-target timeout: max 15s
- Overall deadline: 45s
- Response truncado a 256 KB
- Excerpt mostrado: primeros 1 KB

SURICATA RULES + QUICK-BLOCK

CAPÍTULO 08

`/rules` permite gestionar el file `suricata-rules/suricata.rules` sin tocarlo a mano. Solo operator/admin.

VISTA

1. **Header:** path del file, count de rules, botón "+ new rule"
2. **Suricata control bar:** status del socket (reachable / offline), botón "► reload now", toggle "auto-reload on save"
3. **Templates dropdown:** 5 reglas pre-cargadas (SQLi UNION SELECT, path traversal, /etc/passwd, shell metachars, drop sqlmap UA)
4. **Tabla de rules:** enable-toggle, sid, action (color-coded: drop=rojo, alert=violeta), msg, edit/delete

CREAR REGLA

Click "+ new rule" o pick un template → se abre form con textarea. Pegás Suricata raw rule:

```
alert http any any -> any any (msg:"path traversal"; flow:to_server;  
content:"../"; http_uri; metadata: tag path_traversal; rev:1;)
```

Si no incluí `sid:N;`, w4rya te auto-asigna uno desde 1.000.000. Si los parens están desbalanceados (rule pegada mal), el save rechaza con error (no quiere splicear sid en medio de un content).

ENABLE / DISABLE / DELETE

Toggle en cada fila. "Disable" prepone `#` a la línea (convención Suricata). "Delete" la elimina por completo.

QUICK-BLOCK DESDE UN FLOW

En `/flow/<id>`, al lado del `src_ip`, hay un botón **"block"**. Confirm → genera automáticamente:

```
drop ip <src_ip> any -> any any (msg:"w4rya:block:<src_ip>@<ts>";  
metadata: tag blocked; sid:<auto>; rev:1;)
```

Visual feedback: `✓ blocked` verde por 3s, después vuelve a "block".

AUTO-RELOAD

Activando el toggle **auto-reload on save**, cualquier CRUD de rules dispara también `reload-rules` sobre el socket unix de Suricata (`/var/run/suricata/suricata-command.socket`). El socket lo expone Suricata con el flag `--set unix-command.enabled=yes` (ya wired en el compose).

Si Suricata no está corriendo, el save igual funciona y la respuesta incluye `reload.kind=socket_missing`. La UI muestra el error sin romper el save.

RELOAD MANUAL

Si tenés auto-reload OFF y guardás cambios, podés reloadear cuando quieras con el botón **► RELOAD NOW** del control bar.

ATTACK TIMELINE

CAPÍTULO 09

Vista cronológica de eventos hostiles en `/attacks`. Un "evento" es cualquier flow que:

- Matcheó una regla Suricata (signatures no vacío), OR
- Llevó un flag (tag `flag-out`)

FILTROS

- **Range presets:** last 5 / 10 / 30 / 60 ticks. Default 10.
- **Service dropdown:** filtrar por un service específico, o "all".
- Auto-refresh cada 20s, polling pausado en tabs background.

CADA FILA

COL	CONTENIDO
time	Hora local del flow
type	Chip de color: ✕ ALERT /  LEAK /  + ✕ ambos
attacker	src_ip:src_port
service	Resolved al name configurado
detail	Rule msgs (action color-coded) + flag count si aplica
flow	Link "open →" al detalle

Si no hay servicios configurados o si el range no captura nada, mostrará "no events in range — try a wider tick window".

WAR-ROOM (MODO TV)

CAPÍTULO 10

Pantalla fullscreen pensada para una TV montada al lado del equipo. Sin sidebar, sin header — todo el espacio para datos en tiempo real.

LAYOUT 2x2

PANEL	QUÉ MUESTRA
Services grid	Chips grandes con stats flows/  , color cascada

Incoming attacks	Feed cronológico de los últimos 20 eventos
Top attackers	Bar chart por count, con tally de flags por atacante
Flag leaks	Cards rojas para cada leak, o "✓ no flag leaks" verde si todo OK

TOP BAR

Brand + tick actual + barra de progreso del tick (% completado) + total events + leaks count (rojo glow si >0) + reloj wall-clock.

AUTO-REFRESH

Cada 10s. Polling pausado si la pestaña pierde foco (improbable en modo TV, pero por consistencia).

EXIT

Link "← exit" en footer vuelve a . Esc no cierra (no es modal).

NOTAS Y ALERTAS

CAPÍTULO 11

NOTES POR FLOW

Panel debajo del Meta card en `/flow/<id>`. Cualquier usuario autenticado puede postear, hasta 10000 chars. Útil para coordinar: "este SQLi funciona", "ya tengo exploit para esto", "@alice mirá", etc.

El autor del note se setea automáticamente (session user). Solo el author puede borrar su propia note (admin no puede borrar notas ajenas).

TOASTS GLOBALES

Sistema de notificaciones bottom-right de la pantalla. 4 severidades con color: info (violeta), success (verde), warning (ámbar), danger (rojo). Auto-dismiss después de N segundos o click ✕.

Algunas toasts incluyen un link "open →" que navega directo al flow relacionado.

FLAG-LEAK ALARM AUTOMÁTICA

`FlagLeakWatcher` es un componente background que polea `/query?tags_include=flag-out` cada 15s. La primera vez pobla un "seen set" sin disparar nada (no quiere alertar de leaks históricos). De ahí en más, cualquier nuevo flow con `flag-out` dispara toast danger con link.

El polling se pausa cuando la tab está hidden.

AUDIT LOG

CAPÍTULO 12

`/audit` — solo admin. Append-only tabla de toda acción autenticada que escribe estado:

ACCIÓN	CUÁNDO
auth.login	Login exitoso (incluye role + ip)
auth.login_fail	Password incorrecto (incluye ip + attempt count)
auth.login_blocked	Intento durante lockout por rate-limit
auth.logout	Logout explícito
config.set	PUT /config (incluye qué keys cambiaron)
config.services	PUT /config/services (count)
config.teams	PUT /config/teams (count)
rules.add	Nueva rule (target = sid)
rules.update	Edit (target = sid, details = qué cambió)
rules.delete	Borrado (target = sid)
rules.block_ip	Quick-block (target = ip)
suricata.reload	Reload manual
attack.replay	Exploit testing (target = flow_id, details = target_count + timeout)

FILTROS

- **Actor:** dropdown poblado desde `/audit/actors`
- **Action:** prefix match (auth.*, rules.*, etc)
- **Range:** presets 15m / 1h / 6h / 24h / all

EXPORT CSV

Botón "↓ **export csv**" baja un CSV con los filtros actuales aplicados. Cap 50.000 filas. Filename auto-timestamp UTC.

BACKUPS

CAPÍTULO 13

Script `scripts/backup.sh` snapshot atómico de toda la state operacional:

- `auth/users.yaml` (usuarios + roles + hashes)
- `suricata-rules/*.rules` (todas tus reglas)
- `.env` (con W4RYA_SECRET_KEY — sensible)
- `pg_dump` de las 3 tablas DB: `app_config`, `flow_notes`, `audit_log`

Output: `./backups/w4rya_<UTCISO>.tgz`. Directorio `backups/` gitignored (contiene secrets).

USO

```
./scripts/backup.sh
# o customizando dir:
BACKUP_DIR=/srv/w4rya-bk ./scripts/backup.sh
```

CRON SAMPLE DURANTE CTF

```
# cada 5 min
*/5 * * * * cd /home/user/w4rya && ./scripts/backup.sh >> /tmp/w4rya-bk.log 2>&1
```

RESTORE

```
tar xzvf w4rya_20260524T200000Z.tgz -C /tmp/restore
# revisar los .sql, después:
docker compose exec -T timescale psql -U w4rya -d w4rya < /tmp/restore/app_config.sql
docker compose exec -T timescale psql -U w4rya -d w4rya < /tmp/restore/audit_log.sql
docker compose exec -T timescale psql -U w4rya -d w4rya < /tmp/restore/flow_notes.sql
cp /tmp/restore/users.yaml auth/users.yaml
cp /tmp/restore/*.rules suricata-rules/
docker compose restart api
```

CUIDADO DÓNDE VAN LOS TGZ

El tarball contiene `.env` + bcrypt hashes. Si los compartís off-host, encriptalos primero (`gpg --symmetric`) o usá storage con ACL.

TESTS AUTOMATIZADOS

CAPÍTULO 14

Suite pytest dentro del container api. 29 tests, ~0.3 segundos.

Cubre las pure-function paths que más fácil rompen en silencio:

- `rate_limit`: bucket, expiry, clear
- `app_config.coerce_scalar`: int, bool variants, regex valid/invalid/ReDoS, string passthrough
- `rules.parse_one`: rule válido, disabled, blank, comentario, garbage-preserves-raw
- `rules.inject_sid`: adds-when-missing, no-op-when-present, no-dup-rev, refuses-unbalanced, no-parens
- `rules` round-trip: add+list+toggle+delete sobre tmp file, block_ip happy + quote-injection guard
- `attack`: hex block format/wrap/empty, build_payload empty + client-only filter

CORRERLOS

```
docker compose exec api python -m pytest /app/tests/ -v
```

Salida esperada: `29 passed in 0.3s`.

TROUBLESHOOTING

CAPÍTULO 15

EL API NO ARRANCA

- **"W4RYA_SECRET_KEY env var is required"** → poné un valor en `.env` con `openssl rand -hex 32`
- **"connection to server at ... port 5432 failed"** → timescale tarda 30-60s en bootear la primera vez; los reintentos del api lo recuperan. Si persiste, mirar `docker compose logs timescale`

BUILD DE DOCKER FALLA CON "TEMPORARY FAILURE IN NAME RESOLUTION"

BuildKit no resuelve DNS en algunos hosts. Workaround:

```
docker build --network=host -t w4rya-api:latest \
-f services/api/Dockerfile-api services/api/
```


EL HEALTHCHECK DICE "UNHEALTHY"

```
docker inspect --format='{{.State.Health.Status}}' w4rya-api-1
docker compose logs api | tail -50
```

Si el endpoint `/healthz` devuelve 503, suele ser conn pool a timescale roto. Restartear: `docker compose restart api`.

ME BLOQUEÉ EN /LOGIN

5 fails → 5 min lockout por (ip, username). Esperar, o restartear api para limpiar buckets en memoria: `docker compose restart api`.

FILES ROOT-OWNED DESPUÉS DE SUDO

```
sudo chown -R $USER:$USER auth/ suricata-rules/ backups/
```

SURICATA NO RECIBE RULES NUEVOS

Verificá: (1) suricata está corriendo (`docker compose -f docker-compose-suricata.yml ps`), (2) la file `suricata-rules/suricata.rules` contiene tu cambio, (3) el socket existe (`ls suricata-run/`), (4) reload manual con el botón "► reload now". Si nada, restart suricata.

EL FRONTEND MUESTRA "NO EVENTS / NO SERVICES CONFIGURED"

Setup primer vez: tenés que configurar services en `/config → services`. Los pcaps de ejemplo (en `services/test_pcap`) usan IPs ficticias; mapealas a names en la UI.

ATACANTES VEN MI W4RYA

NO lo expongas a internet. Es para LAN/VPN team-internal. Si DEBE ser remoto: ponelo detrás de un reverse proxy con TLS + setear `W4RYA_COOKIE_SECURE=1` en `.env`. Sin TLS la cookie viaja en claro.

ATAJOS DE TECLADO

CAPÍTULO 16

FlowList sidebar

<code>j</code>	Down
<code>k</code>	Up
<code>w</code>	Scroll a flow actual
<code>x</code>	Star toggle
<code>r</code>	Refetch
<code>i</code>	Toggle filtro <code>flag-in</code>
<code>o</code>	Toggle filtro <code>flag-out</code>
<code>t</code>	Toggle filtro <code>starred</code>

Header

<code>s</code>	Focus search bar
<code>esc</code>	Blur input activo
<code>a</code>	Last 5 ticks
<code>c</code>	Clear time pickers
<code>g</code>	Graph view (Corrie)

FlowView (panel derecho)

<code>h</code>	Prev item
<code>l</code>	Next item
<code>m</code>	Rotar decoder
<code>d</code>	Diff view
<code>f</code>	Load to first diff slot
<code>e</code>	Load to second diff slot

Modal de exploit

<code>esc</code>	Close
<code>tab</code>	Cycle entre botones del modal (focus-trap)

FIN DEL MANUAL

Mantenido en sincronía con la versión del código en `CLAUDE.md`. Si encontrás divergencias, abrir issue en el repo.